CK0255/CKP8377 - Aprendizado Profundo
Lista de Exercícios 1

Prof. Eliezer de Souza da Silva

Entregar até: 5 de Outubro de 2025

Instruções

- Esta lista de exercícios deve ser resolvida individualmente.
- A entrega consistirá em três arquivos (2 arquivos `.pdf` e um arquivo `.ipynb` consistindo em 1 PDF para respostas teóricas, 1 Jupyter notebook e 1 PDF com o código, saídas textuais e gráficas.
    1. Para os exercícios teóricos, as respostas devem ser claras, mostrando o desenvolvimento matemático passo a passo. A entrega deve ser um único PDF com suas respostas.
    2. Para os exercícios de implementação, submeta um único notebook do Jupyter (.ipynb) contendo o código funcional, os resultados (gráficos, saídas) e as análises textuais solicitadas. Submita em conjunto um PDF do notebook completo, com código e todas as saidas e gráficos (pode ser gerado imprimindo a notebook pra formato PDF).
- O exercício tem 3 pontos extras, portanto vale 13 pontos no total. O cálculo da média das lista não muda.
- A entrega deve ser feita através do SIGAA.

Mini-Framework baseada em NumPy

Estes exercícios são baseados no framework modular desenvolvido no laboratório da Aula 4.

Exercício 1: Finalize o Laboratório 1 (3.0 pontos)

Finalize a implementação e as tarefas descritas no notebook em anexo `notebook-lista1.ipynb`.

Exercício 2: Estendendo o Framework NumPy (2.0 pontos)

Estenda o framework que construímos no laboratório.

1. **Novas Ativações:** Crie as classes ‘Tanh‘ e ‘ReLU‘, implementando seus respectivos métodos ‘forward‘ e ‘backward‘.

2. **Softmax e Cross-Entropy:** Crie uma classe ‘MultiClassCrossEntropyLoss‘ e classe ‘Softmax‘ . O método ‘forward‘ deve receber probabilidades normalizadas das classes e calcular o custo (cross-entropy). O método ‘backward‘ deve retornar o gradiente para cada classe, aproveitando a simplificação matemática que vimos em aula.

3. **Classificação Multiclasse:** Utilize as classes ‘MultiClassCrossEntropyLoss‘ e ‘Softmax‘ para implementar um classificar multiclasse e teste no conjunto de dados MNIST. Teste inicialmente com duas classes, depois com três ou quatro (escolha alguns números aleatórios). *Dica:* o notebook do laboratório tem código para carregar os dados do MNIST. Mostre o erro de treinamento e teste, e as curva da loss durante o treinamento.

Exercício 3: Regressão Simbólica (Extra) (1.0 pontos)

O objetivo da regressão simbólica é descobrir a expressão matemática subjacente aos dados. Neste exercício, você usará PyTorch e descida de gradiente para encontrar uma fórmula. Se decidir usar outra framework de diferenciação automática (JAX, a Mini-framework dos exercício anterior), descreva como foi usada.

1. **Dados:** Gere dados a partir de uma função conhecida, e.g., $y = 1.5x^2 - 0.8\sin(x) + \text{ruído}$.

2. **Biblioteca de Funções:** Crie uma biblioteca de funções candidatas:

$$\phi(\mathbf{x}) = [1, x, x^2, \sin(x), \cos(x), e^x, ...]$$

.

3. **Modelo (PyTorch):** Crie um modelo que seja uma combinação linear de todas as funções da sua biblioteca: $\hat{y} = \mathbf{w}^T\phi(x)$. Os pesos $\mathbf{w}$ serão os parâmetros aprendíveis (‘nn.Parameter‘).

4. **Treinamento:** Use o otimizador Adam para treinar os pesos $\mathbf{w}$ minimizando uma função de custo que combina o Erro Quadrático Médio com uma **penalidade L1** sobre os pesos: $J = \text{MSE} + \lambda||\mathbf{w}||_1$. A regularização L1 é crucial para induzir esparsidade (levar pesos irrelevantes a zero).

5. **Análise:** Inspecione os pesos $\mathbf{w}$ aprendidos. Quais pesos têm magnitude significativa? A combinação dos termos com pesos não-nulos se assemelha à função original? Apresente os pesos e a fórmula descoberta.

Teoria e Conceitos Fundamentais

Exercício 4: Conectando Modelos Probabilísticos e Redes Neurais (3.0 pontos)

Este exercício explora a conexão entre a inferência Bayesiana (MAP) e a minimização de risco regularizado.

A) Regressão Linear Bayesiana (Ridge Regression)

Considere um modelo de regressão linear $\hat{y} = \mathbf{w}^T\mathbf{x} + b$.

1. **Verossimilhança (Likelihood):** Assuma que os dados são gerados com ruído Gaussiano, $y_n \sim \mathcal{N}(\mathbf{w}^T\mathbf{x}_n + b, \sigma^2)$. Escreva a função de log-verossimilhança para um conjunto de dados $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$. Mostre que ela é proporcional ao negativo do Erro da Média dos Quadrados ou *Mean Squared Error* (MSE) dado por $\mathcal{L} = \frac{1}{N}\sum_{n=1}^N(\hat{y}_n - y_n)$, com $\hat{y}_n = \mathbf{w}^T\mathbf{x}_n + b$ e $(\mathbf{x}_n, y_n) \in \mathcal{D}$.

2. **Prior:** Assuma uma prior Gaussiana sobre os pesos, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$. Escreva a função de log-prior $p(\mathbf{w})$. Mostre que ela é proporcional ao negativo de um termo de regularização L2 $\lambda\Omega(\mathbf{w}) = \lambda||\mathbf{w}||_2$.

3. **Posterior (MAP):** A estimação do Máximo a Posteriori (MAP) busca os parâmetros que maximizam $p(\mathbf{w}|\mathcal{D}) \propto p(\mathcal{D}|\mathbf{w})p(\mathbf{w})$. Mostre que maximizar o log-posterior é equivalente a minimizar o Risco Empírico Regularizado da Regressão Ridge (L2) dado por $\mathbf{w}^* = \arg\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) + \lambda\Omega(\mathbf{w})$. Que relações temos entre α da priorim, σ^2, e λ da regularização.

4. **Implementação:** Derive o gradiente do negativo do log-posterior em relação a $\mathbf{w}$ e b. Implemente uma solução usando descida de gradiente para encontrar os parâmetros MAP. Demonstre o resultado num conjunto de teste e mostre o gráfico do erro durante o treinamento.

B) Regressão de Poisson Bayesiana

Considere um problema onde queremos prever uma contagem $y_n \in \{0, 1, 2, ...\}$.

1. **Verossimilhança e Função de Ligação:** Assumimos que os dados seguem uma distribuição de Poisson, $y_n \sim \text{Poisson}(\lambda_n)$. Como a taxa λ_n deve ser positiva, usamos uma função de ligação exponencial (log-link): $\lambda_n = \exp(\mathbf{w}^T\mathbf{x}_n)$. Escreva a função de log-verossimilhança.

2. **Posterior (MAP):** Assuma a mesma prior Gaussiana para os pesos, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$. Escreva a expressão para o negativo do log-posterior. Argumente por que este problema não tem uma solução analítica fechada para $\mathbf{w}$.

3. **Implementação:** Derive o gradiente do negativo do log-posterior em relação a $\mathbf{w}$. Implemente uma solução usando descida de gradiente para encontrar os parâmetros MAP para um dataset de contagem sintético que você mesmo irá gerar. Demonstre o resultado num conjunto de teste e mostre o gráfico do erro durante o treinamento.

Exercício 5: Conceitos (1.0 ponto)

Responda brevemente:

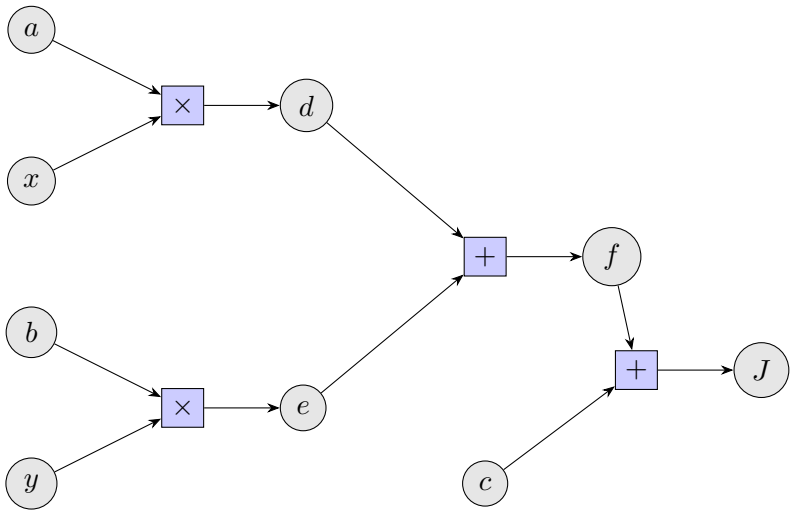
1. O Teorema da Aproximação Universal garante que uma rede neural com uma camada oculta pode aproximar qualquer função. Isso significa que redes profundas (com múltiplas camadas) são redundantes? Explique por que a profundidade é preferível na prática.

2. Explique a conexão entre a Minimização do Risco Empírico Regularizado e o trade-off viés-variância. Como o termo de regularização $\lambda\Omega(\theta)$ afeta este trade-off? O que podemos dizer sobre o papel da priori e o termo de regularização?

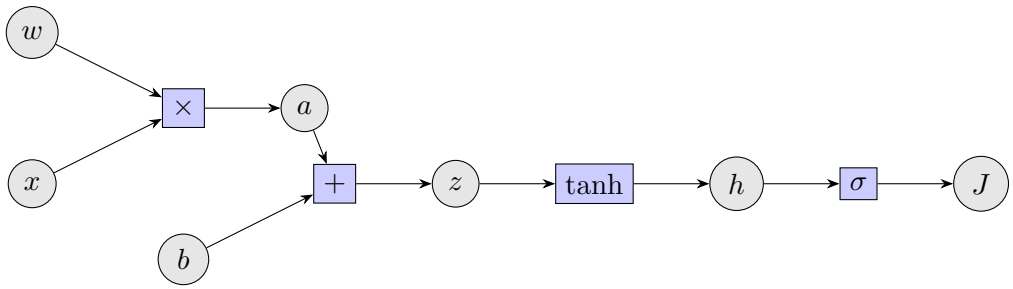
Exercício 6: Derivando Gradientes em Grafos Computacionais (1.0 pontos)

Para os três grafos computacionais abaixo, sua tarefa é aplicar a regra da cadeia (backpropagation) para derivar a expressão analítica do gradiente da saída final (J) em relação a todas as outras variáveis e parâmetros no grafo (e.g., $\frac{\partial J}{\partial a}$, $\frac{\partial J}{\partial b}$, etc.).

1. **Grafo 1: Combinação Linear**



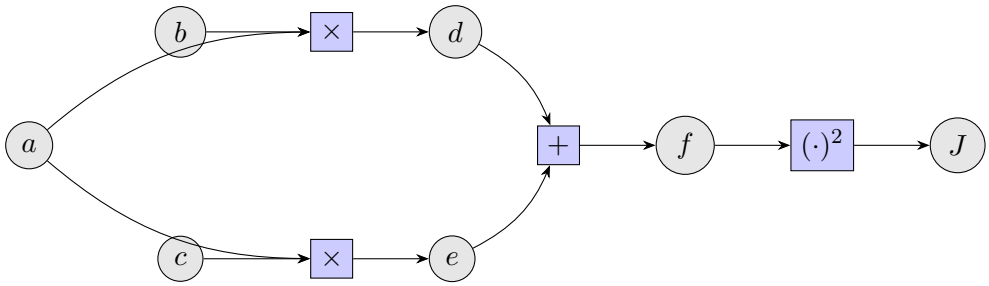
2. **Grafo 2: Neurônio com Ativação Não-Linear**



Lembre-se que $\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$ e $\frac{d}{dz}\sigma(z) = \sigma(z)(1 - \sigma(z))$.

3. **Grafo 3: Variável com Múltiplos Filhos**

Este grafo testa um conceito fundamental do backpropagation: quando uma variável alimenta múltiplos caminhos, seu gradiente total é a **soma** dos gradientes que chegam de cada caminho.



Ao derivar $\frac{\partial J}{\partial a}$ *atenção* em como os gradientes dos caminhos de d e e se combinam.

Exercício 7: Convergência da Descida de Gradiente (Extra) (2.0 pontos)

Este exercício guia você pela prova de que a Descida de Gradiente converge para um ponto estacionário para funções com gradiente L-Lipschitz (L-suaves).

Parte A: Definição e o Lema da Descida

1. Uma função diferenciável $J(\theta)$ é dita **L-suave** se seu gradiente é L-Lipschitz contínuo, ou seja, existe uma constante $L > 0$ tal que:

$$||\nabla J(\theta_1) - \nabla J(\theta_2)||_2 \leq L||\theta_1 - \theta_2||_2, \quad \forall \theta_1, \theta_2$$

   Uma consequência direta desta definição é o **Lema da Descida**, que estabelece um limite superior quadrático global para a função:

$$J(\theta_2) \leq J(\theta_1) + \nabla J(\theta_1)^T(\theta_2 - \theta_1) + \frac{L}{2}||\theta_2 - \theta_1||_2^2$$

   Explique em suas próprias palavras a intuição por trás da condição de L-suavidade. O que ela nos diz sobre a curvatura e geometria da função de custo?

Parte B: Prova de Decrescimento Suficiente A regra de atualização da Descida de Gradiente é $\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$.

2. Substitua $\theta_1 = \theta_t$ e $\theta_2 = \theta_{t+1}$ no Lema da Descida.

3. Em seguida, substitua a regra de atualização da Descida de Gradiente na inequação resultante.

4. Mostre que, após a substituição, você chega à seguinte relação. Obtenha a fórmula para o termo $C(\eta, L)$ que é uma constante que depende apenas de L e η.

$$J(\theta_{t+1}) \leq J(\theta_t) - C(\eta, L)||\nabla J(\theta_t)||_2^2$$

Parte C: A Condição da Taxa de Aprendizado

3. A partir da inequação da Parte B, derive a condição sobre a taxa de aprendizado η que garante que a função de custo não aumente a cada passo (i.e., $J(\theta_{t+1}) \leq J(\theta_t)$). Qual é a escolha de η que maximiza a queda garantida no valor da função de custo a cada passo? [Nesterov(2004)]

Referências

[Nesterov(2004)] Y. Nesterov. *Introductory lectures on convex optimization: A basic course*, volume 87. Springer Science & Business Media, 2004.